

Accelerating Retrieval-Augmented Generation

Derrick Quinn
Cornell University
Ithaca, NY, USA
dq55@cornell.edu

Mohammad Nouri
Cornell University
Ithaca, NY, USA
mn636@cornell.edu

Neel Patel
Cornell University
Ithaca, NY, USA
nmp83@cornell.edu

John Salihu
University of Kansas
Lawrence, KS, USA
jsalihu@ku.edu

Alireza Salemi
University of Massachusetts Amherst
Amherst, MA, USA
asalemi@cs.umass.edu

Sukhan Lee
Samsung Electronics
Hwasung, Republic of Korea
sh1026.lee@samsung.com

Hamed Zamani
University of Massachusetts Amherst
Amherst, MA, USA
zamani@cs.umass.edu

Mohammad Alian
Cornell University
Ithaca, NY, USA
malian@cornell.edu

Abstract

An evolving solution to address hallucination and enhance accuracy in large language models (LLMs) is Retrieval-Augmented Generation (RAG), which involves augmenting LLMs with information retrieved from an external knowledge source, such as the web. This paper profiles several RAG execution pipelines and demystifies the complex interplay between their retrieval and generation phases. We demonstrate that while exact retrieval schemes are expensive, they can reduce inference time compared to approximate retrieval variants because an exact retrieval model can send a smaller but more accurate list of documents to the generative model while maintaining the same end-to-end accuracy. This observation motivates the acceleration of the exact nearest neighbor search for RAG.

In this work, we design Intelligent Knowledge Store (IKS), a type-2 CXL device that implements a scale-out near-memory acceleration architecture with a novel cache-coherent interface between the host CPU and near-memory accelerators. IKS offers 13.4–27.9× faster exact nearest neighbor search over a 512GB vector database compared with executing the search on Intel Sapphire Rapids CPUs. This higher search performance translates to 1.7–26.3× lower end-to-end inference time for representative RAG applications. IKS is inherently a memory expander; its internal DRAM can be disaggregated and used for other applications running on the server to prevent DRAM – which is the most expensive component in today's servers – from being stranded.

CCS Concepts: • **Computer systems organization** → **Parallel architectures**; **Heterogeneous (hybrid) systems**; • **Hardware** → **Hardware accelerators**; • **Information systems** → *Database design and models.*

Keywords: database acceleration, dense retrieval, retrieval-augmented generation (RAG)

ACM Reference Format:
Derrick Quinn, Mohammad Nouri, Neel Patel, John Salihu, Alireza Salemi, Sukhan Lee, Hamed Zamani, and Mohammad Alian. 2025. Accelerating Retrieval-Augmented Generation. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), March 30-April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3669940.3707264

1 Introduction

State-of-the-art natural language processing systems heavily rely on large language models (LLMs)–deep Transformer networks [93] with hundreds of millions of parameters. There is much evidence that information presented in the LLM training corpora is "memorized" in the LLM parameters, forming a parametric knowledge base that the model depends on for generating responses. A major challenge with parametric knowledge is its static nature; it cannot be updated unless the model undergoes retraining or fine-tuning, which is an extremely costly process. This creates a critical issue, especially when it comes to non-stationary domains where fresh content is constantly being produced [108]. Besides, previous studies have indicated that LLMs exhibit limited memorization for less frequent entities [29], are susceptible to hallucinations [83], and may experience temporal degradation [31].

To overcome the challenges presented by LLMs, a potential solution is to enhance them with non-parametric knowledge,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASPLOS '25, March 30-April 3, 2025, Rotterdam, Netherlands
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707264

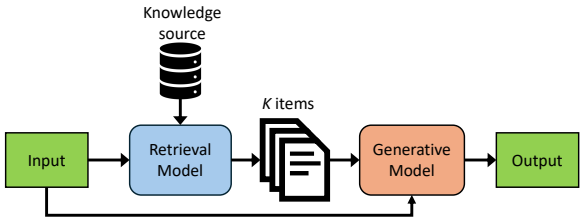


Fig. 1. Overview of the Retrieval-Augmented Generation (RAG) pipeline.

where the LLM is augmented with information retrieved from a knowledge source (e.g., text documents). These approaches have recently gained considerable attention in the machine learning communities [22; 26; 41; 49; 58; 73; 83], and have played key roles in some recent breakthrough applications in the tech industry, such as Google Gemini [90], Microsoft Copilot [86], and OpenAI ChatGPT with Retrieval Plugins [56]. Retrieval-Augmented Generation (RAG) is the term that is used to refer to systems that adopt this approach in the context of LLMs.

A RAG application includes two key components: a retrieval model and an LLM for text generation, called the generative model. When a query is received, the retrieval model searches for relevant items (e.g., documents) and the top retrieved items, together with the input, are sent to the generative model. Current state-of-the-art retrieval approaches use bi-encoder neural networks (called dense retrieval) [30] for learning optimal embedding for queries and documents. Each item is then encoded into a high-dimension vector (called embedding vectors) and stored in a vector database. Such approaches use K-nearest neighbor algorithms for retrieving the top "K" items from the vector database. Figure 1 provides an overview of a RAG application.

The accuracy of generated output in RAG hinges on the quality of the retrieved item list. Conducting an Exact Nearest Neighbors Search (ENNS) to retrieve the precise top K relevant items involves scanning all the embedding vectors in the vector database, which is costly in today's memory bandwidth-limited systems. For example, in a RAG application with a 50GB (using a 16-bit floating point representation) vector database running on an Intel Xeon 4416+ with 8×DDR5-4000 memory channels, and a generative model running on an NVIDIA H100 GPU, ENNS takes up to 97% of the end-to-end inference time (§3).

One strategy to mitigate the retrieval cost is to employ Approximate Nearest Neighbor Search (ANNS), and opt for a faster, but lower-quality search configuration. While lower-quality retrieval can improve search time, our extensive experiments on Question Answering applications demonstrate that a lower-quality search scheme should provide significantly more items to the language model in order to match the end-to-end RAG accuracy of ENNS or a higher-quality, but slower ANNS configuration. This virtually negates any benefits gained during the retrieval phase and even increases the end-to-end inference time.

In this paper, we extensively profile the execution pipeline of RAG, demystifying the complex interplay between various hardware and software configurations in RAG applications [106]. Motivated by the need for high-performance, low-cost, high-quality search and the limitations of current commodity systems, we contribute the Intelligent Knowledge Store (IKS), a cost-optimized, purpose-built CXL memory expander that functions as a high-performance, high-capacity vector database accelerator. IKS offloads memory-intensive dot-product operations in ENNS to a distributed array of low-profile accelerators placed near LPDDR5X DRAM packages.

IKS implements a novel interface atop the CXL.cache protocol to seamlessly offload exact vector database search operations to near-memory accelerators. IKS is exposed as a memory expander that disaggregates its internal DRAM capacity and shares it with vector database applications and other co-running applications through CXL.mem and CXL.cache protocols. Instead of building a full-fledged vector database accelerator, IKS co-designs the hardware and software to implement a minimalist scale-out near-memory accelerator architecture. This design relies on software to map data into the internal IKS DRAM and scratchpads while performing the final top-K aggregation.

In summary, we make the following contributions:

- We demystify RAG by profiling its execution pipeline. We explore various hardware, system, and application-level configurations to assess the performance and accuracy of RAG.
- We demonstrate that RAG requires high-quality retrieval to perform effectively; nonetheless, current RAG applications are bottlenecked by a high-quality retrieval phase.
- We introduce Intelligent Knowledge Store (IKS), which is a specialized CXL-based memory expander equipped with low-profile accelerators for vector database search. IKS leverages CXL.cache to implement a seamless and efficient interface between the CPU and near-memory accelerators.
- We implemented an end-to-end accelerated RAG application using IKS. IKS accelerates ENNS for a 512GB knowledge store by 13.4–27.9×, leading to a 1.7–26.3× end-to-end inference speedup for representative RAG applications.

2 Background

2.1 Information Retrieval in RAG

Recent advancements in RAG indicate superior outcomes when employing dense retrieval over other methods, for unimodal [25; 26; 41] and multi-modal [19; 71; 74] scenarios. Consequently, our emphasis in this study centers on dense retrieval models, exploring their efficiency-related aspects.

In the context of dense retrieval, a query encoder, denoted as E_q, and a document encoder, denoted as E_d, are trained to encode queries and documents, respectively, and map them into a high-dimensional vector space. The similarity score between a document[1] d and a query q is calculated as $s_d = E_q(q) \cdot E_d(d)$, where $E_q(q) \in \mathbb{R}^h$, $E_d(d) \in \mathbb{R}^h$ and h is the hidden dimension of query and document encoders. Then documents are sorted based on their similarity scores and top documents are retrieved [30]. In a real RAG implementation, in an offline phase, all the documents are encoded into embedding vectors. The embedding vectors are stored in a vector database for dense retrieval. In the paper, we refer to the encoded documents as *embedding vectors* and the vectors generated by the retriever model as *query vectors*.

For dense retrieval, two distinct search algorithms are prevalent: Exact Nearest Neighbor Search (ENNS) and Approximate Nearest Neighbor Search (ANNS). ENNS exhaustively computes the complete pairwise distance matrix between embedding and query vectors. In ANNS, however, strategies such as Product Quantization (PQ) [28], Inverted File with Product Quantization (IVFPQ) [7], and Hierarchical Navigable Small World (HNSW) [52], are employed to reduce the search space, seeking to trade off a small amount of search accuracy for higher search efficiency.

2.2 Applications of RAG

RAG has proven beneficial for various tasks in natural language processing [36; 44; 108], including dialogue response generation [2; 8; 10; 83; 91; 92; 96; 97; 99], machine translation [18; 21; 102; 109], grounded question answering [25; 26; 41; 66; 67; 76–78; 85; 107], abstractive summarization [58; 64], code generation [20; 49], paraphrase generation [32; 89], and personalization [37; 72; 73; 75]. Additionally, RAG's application extends to multi-modal data tasks like caption generation from images, image generation, and visual question answering [11; 12; 15; 19; 70; 71; 80].

It is noteworthy that commercial LLM systems employing RAG are typically proprietary, and as such, their implementations are not openly accessible. Nevertheless, insights into the implementation of these systems can be gleaned from open-source releases by research labs within commercial entities. We adhere to a methodology akin to the approach outlined by Izacard and Grave [26] and Lewis et al. [41], both of which are contributions from Meta AI. Our implementations closely align with the depicted pipeline in Figure 1. Specifically, we employ a dense document retrieval model as the retriever and leverage a language model for answer generation, consistent with the aforementioned work. Additionally, for efficient vector search capabilities, we utilize the Faiss [27] library, similar to the aforementioned works.

[1]The term "document" refers to any retrievable item from the knowledge source.

3 Demystifying RAG

In this section, we profile the end-to-end execution of three representative long-form question-answering RAG applications and quantify both the execution time breakdown and the generation accuracy of RAG with different hardware and software configurations: `FiDT5`, where we use the T5-based Fusion-in-Decoder [26; 68] as the generative model, as well as `Llama-8B`, and `Llama-70B`, where we use 4-bit-Quantized Llama-3-8B-Instruct and Llama-3-70B-Instruct [3] as the generative models, respectively. The knowledge source for all workloads is Wikipedia, and a trained BERT base (uncased) model is used to generate embedding vectors for documents. We assume 16-bit number format and test with various vector database sizes (corpus size) that store the embedding vectors. The documents themselves are stored in the CPU memory.

In `FiDT5`, the documents are presented via the Fusion-in-Decoder approach, where documents are encoded by the encoder stage of a T5 model, and these encoded representations are combined for the decoder stage. In `Llama-8B` and `Llama-70B`, retrieved documents are presented as plaintext in the prompt. For more information about the methodology, see Section 6.

In the following subsections, we discuss both the accuracy of an end-to-end RAG system and the retrieval model on its own. *Retrieval accuracy* is discussed in terms of recall, where ENNS is considered to be perfect, and the recall score of an ANNS algorithm is the proportion of relevant documents retrieved by both ENNS and ANNS algorithm compared to the total number of relevant documents retrieved by ENNS. *Generation accuracy* refers to how well an end-to-end RAG system answers questions. For details on the evaluation of generation accuracy, see Section 6.2.

3.1 Tuning RAG Software Parameters

Both the retrieval phase and generation phase of the RAG systems that we use offer support for batching of queries in order to improve data reuse and to amortize data movement overheads over several queries. However, batching is not always an option in practice, particularly in the case of latency-critical applications. We consider batch sizes of 1 for latency-critical uses across all applications and 16 for throughput-optimized applications. Batch size does not impact generation accuracy and only affects execution time. An important parameter in RAG is "K" or the *number of documents* retrieved and fed to the language model for text generation. Increasing the document count significantly impacts the generation time. The computation required for transformer inference scales at least linearly with the input size [93], and if we concatenate the retrieved documents, we face significant computation and memory overhead [14; 112]. In particular, the memory required to store a key-value cache entry for a single token can be computed as follows: $n_{\text{layers}} \times n_{\text{KV-heads}} \times d_{\text{head}} \times n_{\text{bytes}} \times 2$, where n_{bytes} refers to the size of the number format [46]. For `Llama-8B`

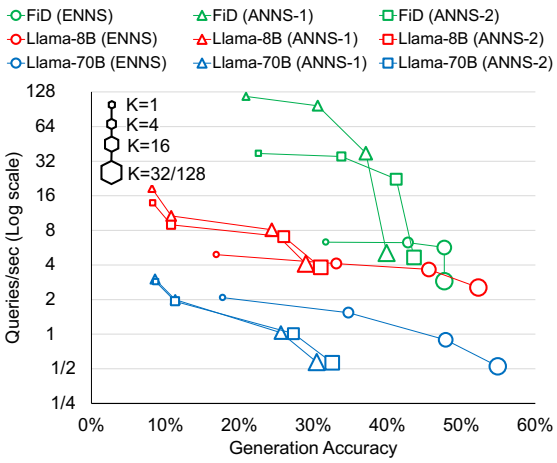


Fig. 2. Generation accuracy vs. throughput (Queries/sec) of representative RAG applications for various retrieval algorithms and document counts (K). The corpus size is set to 50 GB and batch size to 16.

with a 16-bit number format, this is $32 \times 8 \times 128 \times 2 \times 2 = 131$ kB per token. While exact token counts depend on the tokenization process, each document (for all applications) is 100 words long; for `Llama-8B` and `Llama-70B`, this averaged 127 tokens per document across our evaluation dataset.

3.2 Examining Approximate Search for RAG

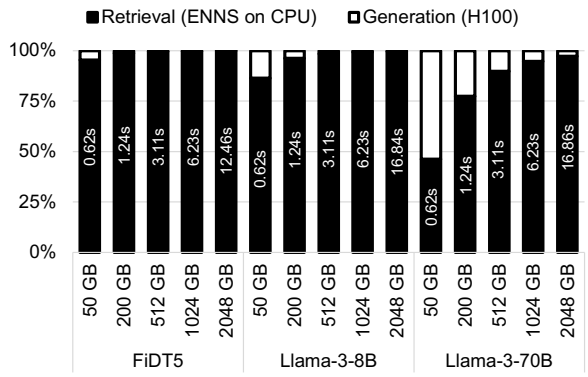
An important algorithmic consideration that can impact the inference time and generation accuracy of RAG is the choice of retrieval algorithm from the vector database, where we can use exact nearest neighbor search (ENNS) or approximate nearest neighbor search (ANNS). The particular algorithm used for retrieval is implemented by a data structure called an *index*, which stores the embedding vectors computed offline, as described in Section 2.1. For ENNS, an index is a wrapper around an array of embedding vectors sequentially iterated over during the search, but for ANNS, the index can be more complex. For example, HNSW stores embedding vectors in a graph-based data structure [52].

To evaluate ANNS, we use the state-of-the-art HNSW [52] ANNS algorithm, and fine-tune the *M* and *efConstruction* parameters to maximize retrieval accuracy while maintaining a reasonable graph, yielding an index with *M* of 32 and *efConstruction* of 128. From this, we evaluate two configurations, ANNS-1 and ANNS-2, which use different *efSearch* parameters: 2048 and 10000. In the context of an end-to-end RAG system, the trade-off of generation accuracy and runtime was evaluated for this index for various choices of *efSearch*. A lower *efSearch* provides higher search throughput, but lower generation accuracy, and a higher *efSearch* provides lower search throughput, but higher generation accuracy. Other HNSW and IVFPQ indexes were tested but provided lower generation accuracy, or similar runtime to ENNS (or even worse, in some cases), negating the benefits of approximation.

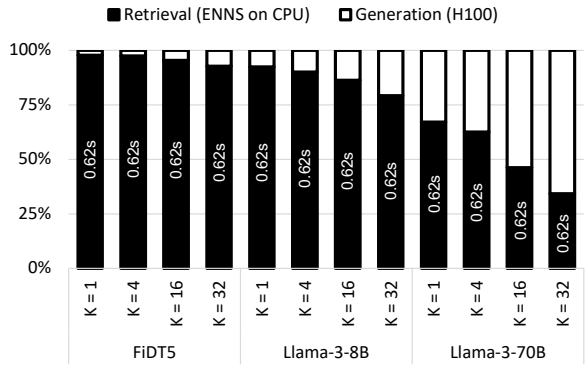
Generation Accuracy with ANNS vs. ENNS: Figure 2 compares the generation accuracy and throughput of ANNS- and ENNS-based RAG applications for `FiDT5`, `Llama-8B`, and `Llama-70B`. The figure illustrates that retrieval quality strongly influences the end-to-end generation accuracy. As shown in Figure 2, with document count of one, compared to ENNS, the generation accuracy of ANNS-1 and ANNS-2 drops by 22.6 and 34.0% for `FiDT5`, 52.8 and 53.4% for `Llama-8B`, and 51.0 and 51.5% for `Llama-70B`, respectively. With a document count of 16, a similar trend in generation accuracy is observed, with ANNS-1 and ANNS-2 leading to an accuracy reduction of 13.6 and 22% for `FiDT5`, 38.4 and 42.2% for `Llama-8B`, and 38.4 and 45.2% for `Llama-70B`, respectively. Interestingly, the impact of retrieval quality on generation accuracy appears to be even larger when using large models that have not been fine-tuned for this task.

Several prior works [6; 110] demonstrate that hyper-parameter tuning can enhance the retrieval accuracy of ANNS, potentially matching that of ENNS across various workloads. While we optimized our HNSW indexes for accuracy and throughput, these indexes could not match ENNS in end-to-end generation accuracy while achieving significantly (more than 2×) faster search. By using a small *efSearch* value, retrieval speed improves significantly, allowing for the use of a larger value of K to compensate for the reduced retrieval quality. However, trading retrieval quality for retrieval speed in this way resulted in lower generation accuracy and end-to-end throughput compared to a larger *efSearch*, where a higher-quality, slower search scheme permits greater accuracy at lower K values (thus reducing generation times). For example, ANNS-2 with 16 documents have 3% higher accuracy and 128% higher throughput compared to ANNS-1 with 128 documents for `FiDT5`. Further improving retrieval quality via exact search gives ENNS-based RAG Pareto-superiority above sufficiently high accuracy thresholds (~43%, ~27%, and ~14% for `FiDT5`, `Llama-8B`, and `Llama-70B`, respectively) as demonstrated in Figure 2. In general, our findings highlight the potential for reducing generation time by leveraging high-quality retrieval methods when high accuracy is required.

Scaling of ANNS and ENNS: Previous works [6; 52] identified the trade-off between retrieval quality and runtime, and challenges with high-quality ANNS have motivated accelerators such as ANNA [40] and NDSearch [95]. While lower-quality ANNS algorithms could possibly provide orders of magnitude faster nearest neighbor search compared with ENNS, high-quality ANNS algorithms are shown to provide only a modest speedup [45; 94]. For example, ANNS-2, which is the best performing ANNS configuration in Figure 2, offers only a 2.5× speedup compared with ENNS. In fact, all the Pareto frontier configurations that provide high generation accuracy in Figure 2 are ENNS. Therefore, in the rest of this section, we focus on understanding how to optimize and accelerate RAG applications with ENNS.



(a) Sensitivity to corpus size. All configurations use K=16.



(b) Sensitivity to K. All configurations use a 50 GB corpus.

Fig. 3. Latency breakdown of `FiDT5`, `Llama-8B`, `Llama-70B` for various values of K, corpus sizes. All configurations use batch size 1. Retrieval is ENNS and runs on CPU, generation runs on a single NVIDIA H100 (SXM) for all generative models. The value in each bar shows the absolute retrieval time.

3.3 End-to-End RAG Performance with ENNS

In this subsection, we profile time-to-interactive (also known as time to first token) [87] for the `FiDT5`, `Llama-8B`, and `Llama-70B` RAG applications and report latency ratios for the retrieval and generation phases. For all experiments, retrieval uses ENNS and runs on the CPU, while generation runs on a single NVIDIA H100 GPU. We select CPU as the baseline for ENNS retrieval, rather than GPU. This decision is made based on the high cost of using GPU memory

As we discussed in Section 3.2, the generation accuracy of RAG applications directly depends on the retrieval accuracy. However, as shown in Figure 3a, utilizing ENNS for retrieval can quickly become an end-to-end bottleneck in RAG applications, even for large models. Although it is possible to compensate for the retrieval accuracy by increasing K (in case of using ANNS), as shown in Figure 3b, increasing K would increase the generation time and is costly in terms of time to first token.

The two phases in a RAG pipeline have different characteristics: ENNS is extremely memory bandwidth-bound, and generation is relatively compute-bound. Nevertheless, the current state-of-the-art focus in building AI systems is only on accelerating the generation phase [1; 5; 17; 34; 48; 53; 63; 82; 100; 101; 104]. Next, we discuss the feasibility of accelerating high-quality nearest neighbor search for future RAG applications.

Batch Size	1		16	
Corpus Size	50 GB	512 GB	50 GB	512 GB
CPU	1	1	1	1
AMX	1.05	1.02	1.10	1.09
GPU	5.2	36.9	6.0	43.7

Table 1. Speedup of Intel AMX and GPU for ENNS, relative to a CPU baseline. AMX speedup is flat for very small batch sizes, due to the memory-bound nature of similarity search. For 50GB and 512GB corpus size, 1 and 8 H100 GPUs are used, respectively.

3.4 High-Quality Search Acceleration

Given the sensitivity of RAG generation accuracy, latency, and throughput to the retrieval quality, it is imperative to focus exclusively on accelerating the retrieval phase of future RAG applications. In this subsection, we discuss the feasibility of accelerating high-quality ANNS and ENNS.

Acceleration of High-Quality ANNS: High-quality ANNS can be as slow as ENNS [45]. There are prior works aimed at building hardware accelerators for high quality ANNS [40; 95] because GPUs are not effective at accelerating key ANNS algorithms such as IVFPQ and HNSW [27]. Unfortunately, the complex algorithms and memory access patterns used for ANNS algorithms also make ANNS accelerators highly task-specific; for example, ANNA [40] and NDSearch [95] can only accelerate PQ-based and graph-based ANNS algorithms, respectively. However, our experimental results, which are in line with prior findings [94], show that different corpora are amenable to different ANNS algorithms.

Acceleration of ENNS: ENNS can be accelerated using conventional SIMD processors such as GPUs and Intel AMX because the algorithm is simple and data-parallel. Table 1 compares the speedup of AMX and GPU against a CPU baseline. Although GPUs can significantly speed up ENNS, as the corpus size increases, the cost of offloading ENNS to GPUs increases significantly. For example, to fit the 50 GB and 512 GB corpus sizes tested in Table 1, we need to use 1 and 8 H100 GPUs, respectively. One of the key contributors to the cost of GPUs is the high-bandwidth memory (HBM) used to implement GPU main memory, which is several times more expensive than DDR or LPDDR-based memories [54]. Lastly, GPUs provision huge amounts of compute relative to memory bandwidth[2], meaning that a large GPU die is poorly utilized by the primarily memory-bound workload of ENNS [24].

[2]NVIDIA H100 80GB provisions 296 Flops/Byte and 592 Int8 Ops/Byte

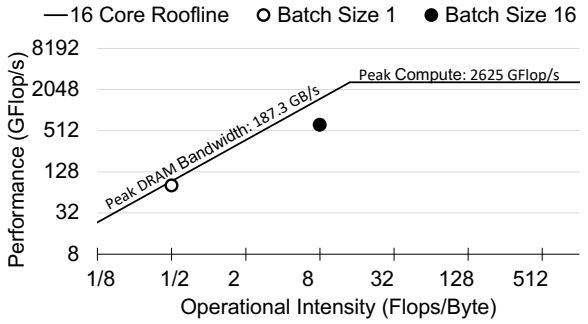


Fig. 4. Roofline model for ENNS using Batch Size 1 and 16. See Section 6 for the experimental setup.

3.5 Summary

The analysis presented in this section, using various software and system configurations for RAG applications, led to the following takeaways:

- Generation accuracy, time to interactive, and throughput of RAG applications can be improved by using a slower but higher-quality retrieval scheme.
- When high-quality retrieval is used, the retrieval phase accounts for a significant portion of end-to-end runtime, regardless of whether the search is performed via ENNS or high-quality ANNS.
- Using GPUs to accelerate ENNS is expensive, and GPUs are not able to accelerate high-quality ANNS effectively or affordably.
- New accelerators for ANNS are highly complex and task-specific due to the unique requirements of ANNS algorithms, while ENNS relies on a very simple scheme, making ENNS simpler to accelerate than ANNS.

4 Case for Near-Memory ENNS Acceleration

ENNS is characterized by the following features:

- ENNS operations exhibit no data reuse for pair-wise similarity score calculations between corpus vectors and a query vector.
- ENNS operations consist of simple vector-vector dot-products coupled with top-K logic.
- ENNS has a regular and predictable memory access pattern.
- ENNS is highly parallelizable, allowing the corpus to be distributed across different processors with a simple aggregation of top-K similarities at the end.

These features make ENNS a prime candidate for near-memory acceleration due to the following reasons: (1) Deep cache hierarchies are not beneficial for ENNS and can even cause slowdown due to the complex cache maintenance and coherency operations managed by the hardware. This is evident from the roofline model in Figure 4 as ENNS running on the CPU cannot saturate the available DRAM bandwidth. (2) The limited data reuse with huge data set size enables low overhead software-managed cache coherency implementation between the host CPU and near-memory accelerators. (3) The regular memory access pattern of ENNS enables coarse-grain virtual to physical address translation on near-memory accelerators. (4) ENNS operations can be efficiently offloaded to a distributed array of near-memory accelerators that each operate in parallel on a shard of corpus data with a low-overhead top-K aggregation phase at the end.

Leveraging these unique features, we design, implement, and evaluate Intelligent Knowledge Store (IKS), a memory expander with a scale-out near-memory acceleration architecture, uniquely designed to accelerate vector database search in future scalable RAG systems. IKS is designed with three requirements in mind: (1) The memory capacity of IKS should be cost-effective and scalable because the size of vector databases for RAG applications is several tens or hundreds of gigabytes and is likely to increase. (2) The near-memory accelerators should be managed in userspace as the cost of context switches and kernel overhead would reduce the benefits of offloads. (3) The near-memory accelerators and host CPU should implement a shared address space; otherwise, explicit data movements between the CPU and near-memory accelerator address spaces will negate the benefits of near-memory offloads; another issue that GPU acceleration of ENNS suffers from. Moreover, a partitioned address space requires rewriting the entire vector database application, as ENNS is just one operation we want to accelerate near the memory, while other data manipulation operations, such as updates, should be managed by the host CPU.

We designed IKS, a type-2 CXL memory expander/accelerator, to meet all these requirements. Our rationale for choosing CXL over DDR-based (or DIMM-based) [4; 35; 62; 111] near-memory processing architecture is that DIMM-based near-memory processing (1) requires sophisticated mechanisms to share the address space between near-memory accelerators and the host [61], (2) limits per-rank memory capacity and compromises the memory capacity of the host CPU when used as an accelerator, and (3) has limited compute and thermal capacity. Instead, IKS relies on asynchronous CXL.mem and CXL.cache protocols to safely share the address space and independently scale the local and far memory capacity of the host CPU, implement a low-overhead interface for offloading from the userspace, and eliminate the limitations on the compute or thermal capacity of the PCIe-attached IKS card. In Section 5, we explain the architecture of IKS and its interface to the host CPU, and how we used it to accelerate end-to-end RAG applications.

5 Intelligent Knowledge Store

5.1 Overview

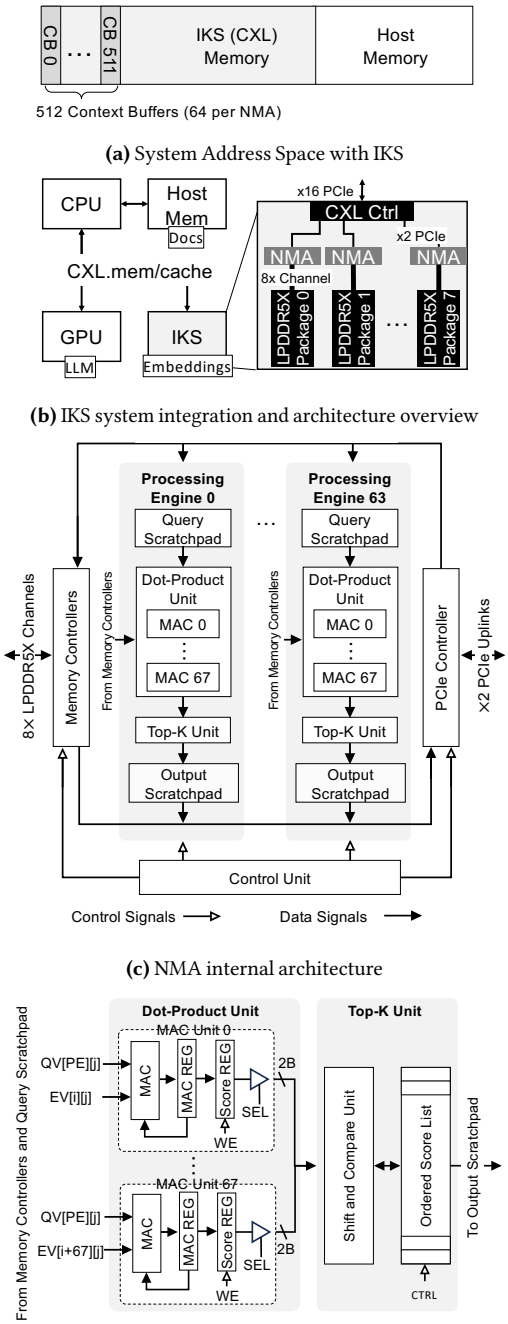
Figure 5 provides an overview of the Intelligent Knowledge Store (IKS) architecture. IKS incorporates a scale-out near-memory processing architecture with low-profile accelerators positioned near the memory controllers of LPDDR5X packages. While IKS can function as a regular memory expander, it is specifically designed to accelerate ENNS over the embedding vectors stored in its LPDDR5X packages.

As shown in Figure 5b, IKS utilizes eight LPDDR5X packages, each directly connected to a Near-Memory Accelerator (NMA) that implements both LPDDR5X memory controllers and accelerator logic. Each package contains 512Gb LPDDR5X DRAM with eight 16-bit channels, similar to CXL-PNM [57] and MTIA [16]. One of the key differences between IKS and these architectures is the *scale-out* near-memory acceleration architecture. IKS distributes the NMA logic over multiple chips, each providing high-bandwidth and low-energy access to its local LPDDR5X package.

Why Scale-Out NMA Architecture? The rationale for such a scale-out NMA architecture is to keep the area of the NMA chip in check. Because memory PHYs are only implemented at the shoreline of a chip [16; 51; 60], to implement 64 LPDDR5X memory channels, we need a chip with an approximate perimeter of 160 *mm*. This is because each LPDDR5X channel PHY approximately occupies a shoreline of 2.5 *mm*, based on the die shots of Apple M2 [59] in 5nm technology. A square-shaped chip with a 160 *mm* perimeter has an area of 1600 mm^2, which is larger than the state-of-the-art lithography reticle limit [98]. Although we can technically manufacture such a large accelerator using chiplets, the area of this huge multi-chip module would be wasted, as it is much larger than what is needed to implement the NMA logic, memory controllers, and PCIe/CXL controllers. For context, the area of an H100 GPU is 814 mm^2.

Splitting the NMAs into smaller chips increases the aggregate chip shoreline and improves yield. Using one NMA per LPDDR5X package requires only eight LPDDR5X memory channels per NMA, necessitating a minimum chip perimeter of 20 *mm*. IKS implements ×2 PCIe 5.0 to provide a 8 GBps uplink connecting each NMA to the CXL controller. With this design, the uplinks to the CXL controller are oversubscribed. Nevertheless, this oversubscription is neither a bottleneck for IKS operating in acceleration mode nor for IKS operating in memory expander mode. In acceleration mode, the bandwidth of local LPDDR5X channels is utilized for dot product calculations, and in memory expander mode, the data is interleaved over multiple LPDDR5X packages and read in parallel over the multiple ×2 PCIe uplinks.

IKS is a type 2 CXL device. IKS's internal memory is exposed as host-managed device memory where both the CPU and IKS can cache addresses within this unified address space (Figure 5a). IKS leverages the low-latency accesses of CXL.mem and CXL.cache protocols to implement a novel interface between the near-memory accelerators and the CPU that: (1)



(a) System Address Space with IKS

(b) IKS system integration and architecture overview

(c) NMA internal architecture

(d) Dot-product unit and top-K units

Fig. 5. (a) IKS internal DRAM, scratchpad spaces, and configuration registers are mapped to the host address space. The scratchpad and configuration register address ranges are labeled as Context Buffers (CB). (b) IKS is a compute-enabled CXL memory expander that includes eight LPDDR5X packages with one near-memory accelerator (NMA) chip near each package. (c) Each NMA includes 64 processing engines. (d) Dot-product units reuse the query vector (QV) dimension across 68 MAC units.

eliminates the need for DMA setup and buffer management, and (2) eliminates the overhead of interrupt and polling for implementing notifications between the CPU and near-memory accelerators (§5.3).

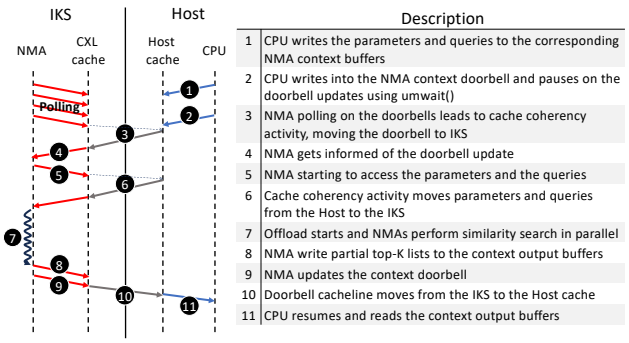
IKS supports spatial and coarse-grain temporal multi-tenancy. In spatial multi-tenancy, the IKS driver partitions embedding vectors that belong to different vector databases across different packages, allowing each NMA to execute ENNS independently per vector database. For temporal multi-tenancy, the IKS driver time-multiplexes similarity search in NMAs among different vector databases that store their embedding vectors in the same LPDDRX5 package. Time multiplexing takes place at the boundary of a complete similarity search.

Why LPDDR? For IKS, a customized type-2 CXL device that should support cost-effective high capacity, neither HBM (expensive) nor DDR (general-purpose) are good options. LPDDR DRAM packages are integrated as part of system-on-chip designs, resulting in shorter interconnections, faster clocking, and less power wastage during data transmission. The most recent release of LPDDR, LPDDR5X, offers a bandwidth of 8533 Mbps per pin, exceeding that of DDR5, which provides a bandwidth of 7200 MTps. However, one challenge with using LPDDR in a datacenter setting is reliability, as LPDDR was originally designed for mobile systems. Although we could provision an in-line ECC processing block for error detection and correction, ENNS similarity search is resilient to bit flips, and rare bit flips in ENNS have negligible impact on the end-to-end RAG accuracy.

5.2 Offload Model

The IKS address space is shared with the host CPU. The host CPU stores embedding vectors with a specific data layout (that we discuss in Section 5.5) in contiguous physical addresses in IKS, while the actual documents are stored in the host memory (either in DDR memory or CXL memory). The CPU runs the vector database application, which offloads the similarity calculations (i.e., dot-products between the query vectors and embedding vectors) using `iks_search(query)`, a blocking API that does not require a system call or context switch. After each update operation, the vector database application will flush CPU caches to ensure that when `iks_search(query)` is called, IKS does not contain any stale values.

`iks_search(query)` hides the complexity of interacting with IKS hardware from the programmer by writing an *offload context* to IKS and initiates an offload by writing into a doorbell register. The offload context and doorbells are communicated through memory-mapped regions called *context buffers* to the IKS as shown in Figure 5a. An *offload context* includes query vectors, vector dimensions, and the base address of the first embedding vector stored in each LPDDR5X package. The host process then uses `umwait()` to block on the doorbell register (shared between IKS and the host and kept coherent via the CXL.cache protocol) to implement efficient notification between the paused CPU process and near-memory accelerators [105].



	Description
1	CPU writes the parameters and queries to the corresponding NMA context buffers
2	CPU writes into the NMA context doorbell and pauses on the doorbell updates using umwait()
3	NMA polling on the doorbells leads to cache coherency activity, moving the doorbell to IKS
4	NMA gets informed of the doorbell update
5	NMA starting to access the parameters and the queries
6	Cache coherency activity moves parameters and queries from the Host to the IKS
7	Offload starts and NMAs perform similarity search in parallel
8	NMA write partial top-K lists to the context output buffers
9	NMA updates the context doorbell
10	Doorbell cacheline moves from the IKS to the Host cache
11	CPU resumes and reads the context output buffers

Fig. 6. CPU-IKS interface through cache coherent CXL interconnect.

As IKS uses a scale-out near-memory processing architecture (§5.1), the embedding vectors are distributed across different near-memory accelerators' local DRAM. Therefore, after all the near-memory accelerators complete the offload, the CPU process waiting on `umwait()` will be notified and execute an aggregation routine to construct a single top-K list. This top-K list is then used to retrieve the actual top-K documents from the host memory. The CPU will locate documents based on the physical addresses of the top-K embedding vectors, as the addresses of the embedding vectors stored in IKS are known a priori.

5.3 Cache Coherent Interface

IKS leverages the cache-coherent interconnect in CXL.cache to implement an efficient interface between near-memory accelerators and host processes through shared memory. Figure 6 illustrates the transactions through the CXL.cache interface between the host and IKS to initiate and conclude an offload. The host process writes the *offload context* to the predefined *context buffer* address range shared between NMAs and the host CPU (step 1). Note that the context buffer is cacheable, and the CPU uses temporal writes to populate the buffers. Next, the host process writes into a doorbell register, which is mapped to a cache line shared by NMAs. NMAs poll on the doorbell register, and as soon as there is a change, the offload starts (step 4). Once the host updates the doorbell register, it calls `umwait()` to monitor the register for changes from the IKS side.

Before computation in the NMA can start, the NMA reads the offload context from the IKS cache (step 5) and the context written by the host is moved to NMA's scratchpad. Once the NMA computation is complete, the NMA updates the context buffers with the partial list of similarity scores and physical addresses of the corresponding embedding vectors. Lastly, the NMA writes into the doorbell register, and the host gets notified of the completion of the offload through the `umwait()` mechanism (step 11).

Our experimental results on a two-socket Sapphire Rapids CPU show that communicating the offload context through

cache-coherent shared memory provides 1.6× higher throughput compared with using non-temporal writes that mimic the PCIe MMIO datapath (i.e., CXL.io). Using a cache-coherent interconnect to implement the notification mechanism through the producer/consumer-style doorbell register eliminates the need for expensive interrupt or polling mechanisms.

5.4 NMA Architecture

As shown in Figure 5c, each NMA implements 64 processing engines to accommodate similarity score calculations for up to 64 query vectors in parallel. Each processing engine includes a query scratchpad, dot-product unit, Top-K unit, and output scratchpad. There is a central control unit in each NMA that generates memory accesses, controls data movement within the NMA, and activates processing engines based on the number of query vectors provided by the host CPU. The network-on-chip implements a fixed broadcast network from DRAM to all the processing engines to reuse data when multiple processing engines are active and evaluate similarity scores against different query vectors.

As shown in Figure 5d, the dot-product unit includes 68 MAC units, each operating at a 1 GHz frequency and providing 68 GFLOPS (16-bit floating point multiply-accumulate operations) compute throughput; therefore saturating the 136 GBps memory bandwidth of the LPDDR5X channels. Each MAC unit evaluates the similarity score between the query (stored in the query scratchpad) and an embedding vector that is read from DRAM in *VD* (Vector Dimension) cycles. All the processing engines operate on the same data that is read from the DRAM; in other words, each processing engine evaluates the similarity score between different query vectors and the same set of embedding vectors. Therefore, for a batch size of one, only one processing engine is utilized, and for a batch size of 64, all the processing engines are utilized. This way, we reuse the embedding vectors that are read from DRAM across different batch sizes.

As illustrated in Figure 5d, within an active dot-product unit, 68 MAC operations are performed in each clock cycle. The first input of the MAC units is dimension j of the query vector in processing engine PE (QV[PE][j]), and the second input is dimension j of the embedding vectors i to i+67 read from DRAM. As mentioned earlier, it takes *VD* (Vector Dimension) cycles for a dot-product unit to evaluate the similarity score for a block of 68 embedding vectors. Once the similarity score is evaluated, it is loaded into a *score register* (shown in Figure 5d) in the next clock cycle, and the MAC unit gets busy evaluating a new similarity score for the next 68 embedding vector block. The score registers (68 per processing engine) are then streamed out to the Top-K unit in the next 68 clock cycles.

The Top-K unit maintains an ordered list of the scores by comparing the incoming similarity scores with the head of the ordered list. Figure 5d illustrates the Top-K unit. If the value of the incoming score is larger, it is ignored; otherwise, it is inserted into the ordered list. Because the vector dimensions are much larger than 68, the serialized insertion into the ordered list is overlapped with the similarity score evaluations and is not on the critical path of the NMA offload.

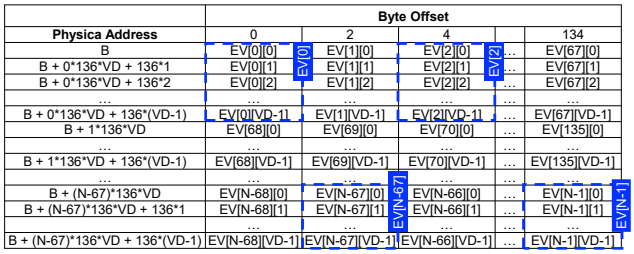


Fig. 7. Data layout inside each LPDDR5X package. The host CPU communicates the base address "*B*", vector dimension "*VD*", and the number of vectors "*N*" to the NMAs for each offload. Four embedding vectors (EVs) are highlighted in this layout.

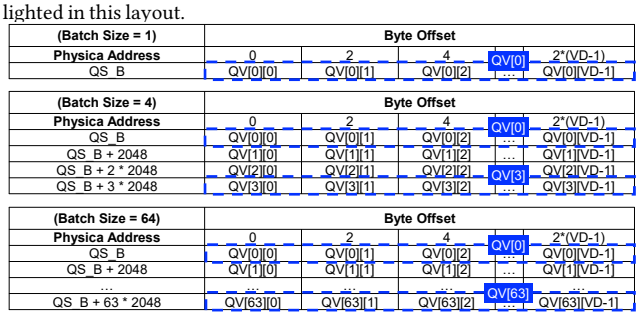


Fig. 8. Data layout inside the query scratchpads mapped to host memory address at query scratchpad base address "QS_B". As we increase the batch size, more query scratchpads are populated with distinct query vectors.

After all the embedding vectors stored in the DRAM are evaluated, the control unit signals the end of the offload by loading the ordered Top-K list into the output scratchpad and writing to the doorbell register. The host CPU is then notified and can read the content of the output scratchpads through the CXL.cache protocol. Note that both the query scratchpads and the output scratchpads are mapped to the host memory address space. In the current incarnation of the NMA, the size of the query scratchpad (per processing engine) is 2KB, and we keep an ordered list of 32 scores (i.e., we set K to 32 in the hardware).

5.5 Data Layout Inside DRAM and Query Scratchpad

The host CPU is required to store the embedding vectors in blocks of 68 vectors, laid out in the DRAM as shown in Figure 7. Because each embedding vector dimension is 2 bytes (16-bit floating point), each block is stored in $136 \times VD$ bytes within DRAM, where *VD* is the vector dimension. Within a block, the embedding vectors are stored in column-major order. This layout allows for efficient batching of corpus vectors, as each may be read and processed dimension-by-dimension. Consequently, each NMA will access up to 136 bytes per cycle from the memory controller read queue, comprising one element from 68 distinct embedding vectors.

As discussed in Section 5.3, the host CPU will fill the query scratchpads with query vectors before an offload starts. The

Platform	Parameter	Description
CPU	CPU model	Intel Xeon 4416+ 16 cores @ 2.00 GHz
	L1 Cache	48 kB dcache, 32kB icache
	L2 Cache	2MB
	L3 Cache	37.5 MB shared
	AVX	2x AVX-512 FMA units (164 GFlop/s/core)
	OS	Ubuntu 22.04.3
	Kernel	Linux 5.15.0-88-generic
	Memory	512 GB DDR5-4000 across 8 channels (256 GB/s)
AMX	–	Intel AMX (BFloat16, 500 GFlop/s/core)
IKS (emulated)	–	1.1 TB/s, 69.9 TFlop/s
GPU	GPU Model	NVIDIA H100 SXM: 3.35 TB/s, 1979 TFlop/s

Table 2. Processing Element Options. Memory configuration for Intel AMX is the same as for CPU.

query vectors are stored in sequential addresses within the query scratchpads, as illustrated in Figure 8.

This data layout inside DRAM and query scratchpads simplifies the address generation as well as the network-on-chip architecture of the NMAs. We modified the memory allocation scheme in the vector database application to implement the block data mapping of embedding vectors inside IKS DRAM as shown in Figure 7.

6 Experimental Methodology

6.1 Experimental Setup

To evaluate the performance of the IKS, we developed a simulator (see appendix A) and fed ENNS traces into it to obtain the retrieval time of IKS. The simulator is a cycle-approximate performance model that utilizes timing parameters from the RTL synthesis, LPDDR5X access timing, PCIe/CXL timing [43; 81], along with calculations of real software stack overhead (top-K aggregation and `umwait()` overhead). It emulates an IKS as a CXL device running on a remote CPU socket. We implemented the end-to-end RAG application described in Section 3 (i.e., `FiDT5`, `Llama-8B`, and `Llama-70B`), including the APIs for distributing queries to the NMA query scratchpad and reducing partial top-32 lists on the CPU. We ran the experiments on two servers equipped with Intel Xeon 4th generation CPUs and one NVIDIA H100 GPU NVIDIA GPUs. The system configuration is shown in Table 2.

We implemented the RTL design of the Near-Memory Accelerator (NMA) used in IKS and synthesized it using Synopsys Design Compiler targeting TSMC's 16nm technology node. This process involved collecting key metrics such as area, power, and timing to ensure the design meets the optimal criteria for operation at 1 GHz. For other components, we estimated the area of the memory controllers and PHYs based on die shots from the Apple M2 chip, which utilizes LPDDR5 in a 5nm process [50]. Since the area scaling of mixed-signal components is negligible [23; 88], we assumed the same area for the LPDDR5X PHYs and memory controllers when scaling to 16nm technology.

We developed a power model by evaluating the energy consumption of processing operations at the RTL level and incorporating the energy required for data access to scratchpads and LPDDR memory. For example, accessing data in SRAM consumes 39 fJ per bit, while LPDDR memory access requires 4 pJ per bit [13]. Since these energy values depend on the underlying technology node, we scaled them to correspond to a 16nm technology node for consistency [79].

6.2 Software configuration

Google's Natural Questions (NQ) dataset [38; 39] is used for the evaluation of models. Meta's KILT benchmark [65] divides these into training (*nq-train*) and validation (*nq-dev*) datasets. For the retrieval phase, we use a BERT base (uncased) model trained to perform similarity searches between questions and their supporting documents in *nq-train*. The document corpus is constructed as described in [30], and an index is created using Faiss [27] to perform the similarity search[3]. Across ENNS and ANNS, Faiss is used for index management. The only change made in our evaluation is the use of Intel's OneMKL BLAS backend for ENNS for all batch sizes, as this provided better performance than the default Faiss search scheme, which uses only BLAS for batch sizes 20 and above.

`FiDT5` Application: For testing the accuracy of `FiDT5`, as described in [26], the generator is initialized as a pretrained T5-base model (220 million parameters), then fine-tuned to predict answers from question-evidence pairs in the *nq-train* dataset.

To evaluate `FiDT5` on the *nq-dev* dataset, we use the exact match metric [69], which normalizes answers and compares them against a list of acceptable answers. For `FiDT5`, *generation accuracy* scores refer to the percentage of *nq-dev* questions for which the RAG application generates a correct answer based on this exact match criterion.

`Llama-8B` and `Llama-70B` Applications: To evaluate `Llama-8B` and `Llama-70B` on the *nq-dev* dataset, we guide the model via prompting and evaluate *generation accuracy* using a Rouge-L "recall" metric [47], which scores answer predictions based on the proportion of the correct answer that is continuously present in the predicted answer. The model is instructed to give a short answer and to answer only if it is "completely sure." The prompting approach is used over fine-tuning to reflect an implementation that preserves the generality of the models. However, the downside of this approach is that evaluation is limited by prompt adherence, which is why the "recall" metric is used over precision or F1-Score. When evaluating end-to-end RAG systems, the applications process a batch of queries by first performing retrieval, then generation, before processing the next batch.

7 Experimental Results

7.1 Effectiveness and Scalability of IKS Retrieval

[3]We adapt the Faiss implementation of ENNS by using Intel MKL as the BLAS backend, using BLAS for all batch sizes, and increasing the corpus block size from 1024 to 16384. See appendix A.

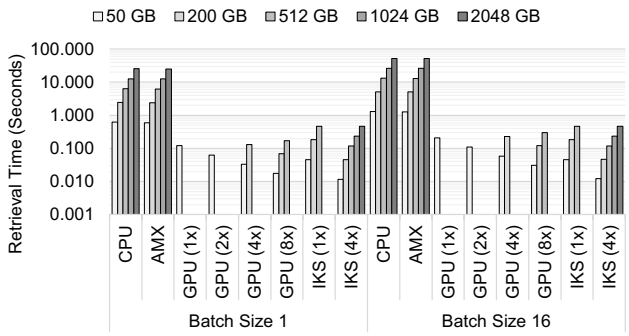


Fig. 9. Comparison of ENNS retrieval time for CPU, AMX, GPU (1, 2, 4, and 8 devices), and IKS (1, and 4 devices) for various corpus sizes. The absence of bars in specific GPU and IKS configurations indicates that the corpus exceeds the capacity of the accelerator memory. The Y-axis is in log-scale.

Figure 9 compares the performance of IKS with CPU, AMX (idealized, based on speedup for matrix multiplication), and GPU ENNS retrieval. IKS provisions compute and memory bandwidth to balance the pipeline at the maximum batch size of 64; as such, performance is almost flat for batch sizes less than 64. As shown, the purposefully built NMA logic for ENNS enables 1 IKS unit to outperform 1 GPU for a 50 GB corpus for batch sizes 1 and 16 by 2.6× and 4.6×, respectively. This counterintuitive speedup of IKS over GPUs, which theoretically have both higher FLOPS and memory bandwidth than IKS, is due to two reasons: (1) top-K tracking and aggregation on GPUs is not efficient, while IKS includes specialized Top-K units; and (2) low utilization of the GPU chip translates to limited memory bandwidth usage, as saturating the entire HBM memory bandwidth requires many streaming multiprocessors and tensor cores to issue memory accesses to DRAM in parallel.

To demonstrate the scalability of IKS, we include the retrieval time of multi-GPU and multi-IKS setups. Because each H100 GPU can fit 80 GB of embedding vectors, 8 GPUs can accommodate maximum corpus size of 640 GB. However, with only four IKS devices, we can fit up to a 2 TB corpus size. As shown in Figure 9, with additional GPUs and IKS units, the retrieval time for the same corpus size decreases, demonstrating the high data-level parallelism of ENNS and the strong scaling of both GPU and IKS. For example, GPU retrieval time for a 50 GB corpus size reduces by 1.9×, 3.6×, and 6.9× with 2, 4, and 8 GPU devices, respectively, and IKS retrieval time for a 50 GB corpus size reduces by 1× and 3.9× with 1 and 4 IKS units, respectively. Due to the low-overhead IKS-CPU interface, the dominance of similarity search latency in end-to-end ENNS retrieval, and the highly parallelizable nature of ENNS, IKS also provides near-perfect weak scaling. For instance, the retrieval time for a 2 TB corpus on 4 IKS units is only 100μs longer than for a 512 GB corpus on 1 IKS unit. However, we do not evaluate configurations with more than four IKS units, and the overhead of host-side final top-K aggregation scales as additional units are added. Additionally, we do not evaluate deployments of IKS spanning multiple nodes.

Table 3 reports the absolute time breakdown of ENNS retrieval on IKS. We break down the retrieval time of IKS into four components: transfer time of query vectors over the CXL interconnect to the NMAs, time for performing dot-products (both computation and DRAM accesses), updating the top-k score lists in parallel on all NMAs, and time for reducing the partial top-32 lists into a single one on the CPU. The retrieval time of IKS does not change with the value of K (with a maximum K value of 32). This is because IKS always returns 32 top similarity scores, and it is up to the retriever model to pass between 1 to 32 of them to the generative model. As shown in the table, the majority of time is spent on computations and DRAM accesses, and the overhead of initiating offload over the cache-coherent interconnect and aggregating top-K documents on the CPU is negligible.

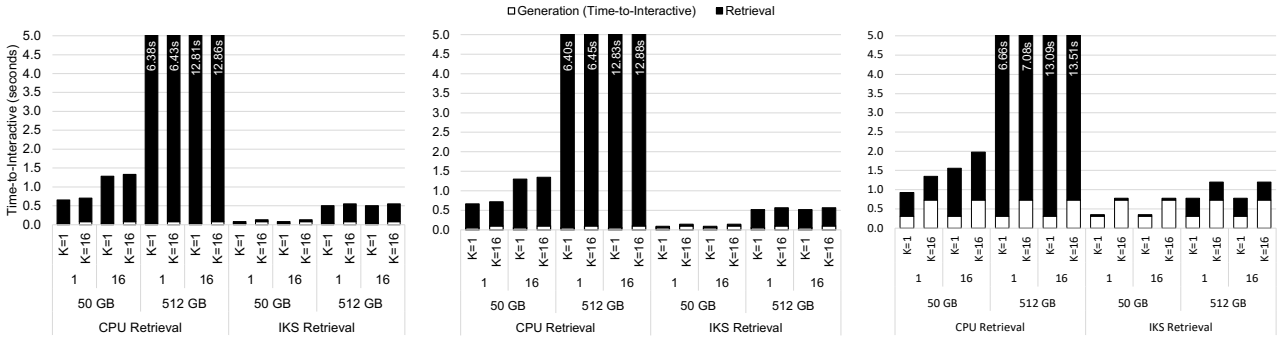
7.2 End-to-End Performance

Figure 10 compares the end-to-end inference time of `FiDT5`, `Llama-8B`, and `Llama-70B` when CPU and IKS are used for ENNS retrieval for various batch sizes, document counts, and corpus sizes. As shown, for large corpus sizes or large batch sizes, the inference time of the RAG applications with CPU retrieval exceeds several seconds, which is not acceptable for user-facing question-answering applications. IKS significantly reduces the ENNS retrieval time for the applications. The end-to-end inference time speedup provided by IKS ranges between 5.6 and 25.6× for `FiDT5`, between 5.0 and 24.6× for `Llama-8B`, and between 1.7 and 16.8× for `Llama-70B` for various batch sizes, corpus sizes, and document counts.

To gain a comprehensive understanding of how the performance and accuracy of RAG applications with IKS acceleration compare across various configurations, Figure 11 depicts the queries per second and accuracy of `FiDT5`, `Llama-8B`, and `Llama-70B` implemented using four different configurations: RAG with ENNS running on CPU, RAG with ANNS (two configurations) running on CPU, and RAG with ENNS running on IKS. The generative model runs on the GPU in all these configurations. As illustrated in Figure 11, although ANNS-2 configurations exhibit higher throughput compared to ENNS (running on the CPU), their accuracy is lower. For RAG applications that use IKS, retrieval is not a bottleneck, and throughput is significantly improved, even compared to ANNS, as the same generation accuracy can be achieved with

Corpus Size	50 GB		512 GB	
Batch Size	1	64	1	64
Write Query Vector	0.3 us	1 us	0.3 us	1 us
Dot-Product	45.96 ms	45.96 ms	470.6ms	470.6 ms
Partial Top-32 Read	0.7 us	22.4 us	0.7 us	22.4 us
Top-K Aggregation	19 us	540 us	23 us	390 us
Total	46.0 ms	46.5 ms	470.6 ms	471.0 ms

Table 3. Breakdown of ENNS latency on IKS.



(a) FiDT5 **(b)** Llama-8B **(c)** Llama-70B

Fig. 10. Inference time breakdown of CPU vs. IKS retrieval for FiDT5, Llama-8B, and Llama-70B. Generative model runs on GPU.

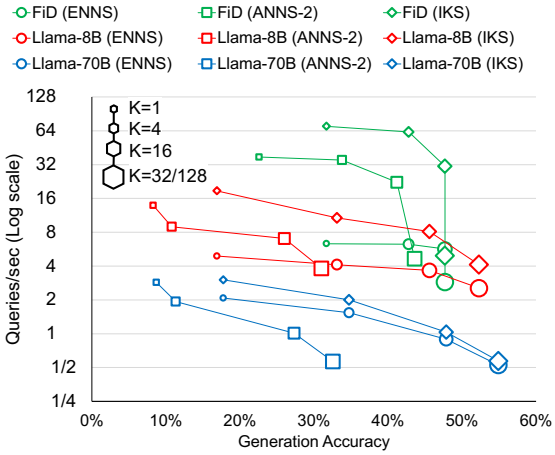


Fig. 11. Comparison of accuracy and throughput of FiDT5, Llama-8B, and Llama-70B for various configurations. ANNS-2 is an HNSW index with *M*, *efConstruction*, and *efSearch* of 32, 128, and 2048, respectively.

smaller values of K (i.e., smaller but more accurate context sent to the generative model).

7.3 Power and Area Analysis

The area of each NMA, which contains 64 processing engines, each comprising a dot-product unit, a 2 KB SRAM query scratchpad, a top-K unit, and an output scratchpad, is approximately 3.4 mm^2 in the 16nm TSMC technology node. Additionally, 14 mm^2 is required for the PHYs and memory controllers. However, the area of the NMA chip is determined by the shoreline because the 21 mm of shoreline required per NMA (20 mm for the LPDDR5X PHYs and 1 mm for PCIe PHYs §5.1) necessitates that the NMA occupy at least 27.56 mm^2 in the 16nm technology node. The NMA can be manufactured using older technology nodes to reduce costs and prevent area wastage, as the PHY area (which is mixed-signal) does not scale at the same rate as SRAM and logic [23; 88].

For a batch size of 1 and vector dimensions of 1024, the processing engines, along with the corresponding query scratchpad accesses, consume approximately 59 mW, while accessing embedding vectors from LPDDR memory requires 4.35 W. As a result, the total power consumption of IKS for a batch size of 1 is 35.2 W. With larger batch sizes, data reuse ensures that the power required for LPDDR access remains constant, but the power consumption of the processing engines increases linearly as more engines are activated to handle the additional workload. For instance, at full utilization with a batch size of 64, the total power consumption increases to 65 W.

7.4 Cost and Power Comparison with GPU

IKS utilizes LPDDR5X memory to store embedding vectors. While figures for the cost of LPDDR5X are not yet available, we assume that HBM is more than 3× more expensive than LPDDR [54]. Since a single IKS unit includes 6.4× as much onboard memory as a single NVIDIA H100 GPU, the memory cost of IKS is expected to be approximately 2.5× greater than that of a GPU.

For the comparison of compute unit cost, the GPU has a die area of 826 mm^2, while the IKS NMAs total a die area of 220 mm^2. Because the production cost of a chip increases superlinearly with die area [55], an IKS unit (with 5× larger memory capacity) is expected to cost a fraction of a GPU.

8 Discussion

IKS provides a cost-effective solution for accelerating ENNS, where the quality of the search is not dataset-dependent. However, if the dataset is amenable to clustering, then the accuracy gap between ENNS and ANNS would reduce, making ANNS more attractive for retrieval. Moreover, IKS is best-suited to RAG applications requiring very high recall, and for datasets difficult to search with existing ANNS schemes with relatively large batch sizes. For example, modern ANNS schemes cannot eliminate more than 99% of the search space for the GloVe dataset [94], so at least 64% of the corpus must be read by an ANNS that does not offer data re-use across queries; in which case the overheads of common ANNS schemes reduces performance to below that of ENNS. However, for datasets that are easier to filter, there is an opportunity for improvement by incorporating approximation techniques into IKS; however, this

introduces significant challenges as IKS owes much of its performance to the sequential memory access pattern of ENNS.

One key inefficiency of IKS is that it performs an exhaustive search over the entire corpus, which consumes energy and saturates memory bandwidth. The high internal memory bandwidth utilization of ENNS can cause slowdowns for external accesses by other applications that use IKS as a memory expander, rather than a vector database accelerator. Exploring early termination of similarity search [9; 42] could be a natural solution for reducing the memory bandwidth utilization of ENNS without compromising search accuracy.

Another inefficiency in the current version of IKS is the low NMA chip utilization for batch sizes less than 64. The rationale for overprovisioning NMA compute is that we effectively have free area on the NMA chip. Note that each NMA chip requires eight LPDDR5X memory channels, which demand 20 mm of chip shoreline. Therefore, the minimum NMA chip area is 25 mm^2 (§5.1). Thus, the area on NMA is effectively free up to a cap of 25 mm^2. We chose to utilize this "free" area to overprovision compute so that IKS remains memory-bandwidth bound for all batch sizes below 64. There are opportunities for circuit-level techniques, such as clock and power gating, to power off extra processing engines when the batch size is below 64. Moreover, dynamic voltage and frequency scaling can be used to reduce the frequency and voltage of the NMA chip for batch sizes less than 64, allowing multiple processing engines to perform similarity searches for each query vector.

9 Related Work

Sim et al. [84] implement a computational CXL memory solution for near-memory processing and showcased ENNS acceleration inside the CXL memory. However, this work implements CXL memory using DDR DRAM, which does not meet the power and bandwidth requirements for ENNS on large corpus sizes used in RAG. Additionally, our work implements a novel interface between host and near-memory accelerators through CXL.cache. Lee et al. [40] and Wang et al. [95] present near-data accelerators for PQ- and Graph-based ANNS, respectively. However, we accelerate ENNS because different corpora are amenable to different ANNS algorithms, and the complex algorithms and memory access patterns of such ANNS schemes also make ANNS accelerators highly task-specific. Ke et al. [33] propose near-memory acceleration of DLRM on Samsung AxDIMM. AxDIMM is based on a DIMM form factor that limits per-rank memory capacity and compromises the memory capacity of the host CPU when used as an accelerator (§4). In contrast, IKS does not strand the internal DRAM space and does not have capacity or compute throughput limitations.

Concurrent with our work, others have also observed that low-quality retrieval can lead to both low-quality and slow generation. Corrective RAG filters out irrelevant documents from the retrieved list before sending them to the LLM [103], while Sparse RAG enables LLMs to use only highly relevant retrieved information [112]. In this work, we used ENNS to eliminate the risk of low-quality retrieval and reduce the context size.

10 Conclusion

In this work, we profiled representative RAG applications and showed that the retrieval phase can be an accuracy, latency, and throughput bottleneck, highlighting the importance of an exact, yet high-performance and scalable retrieval scheme for future RAG applications. We designed, implemented, and evaluated the Intelligent Knowledge Store (IKS), a CXL-type-2 device for near-memory acceleration of exact K nearest neighbor search. The key novelty of IKS is the hardware/software co-design that enables a scale-out near-memory processing architecture by leveraging cache-coherent shared memory between the CPU and near-memory accelerators. IKS offers 18-52× faster exact nearest neighbor search over a 512 GB vector database compared to executing the search on Intel Sapphire Rapids accelerators, leading to 2.0-49× lower end-to-end RAG inference time.

Acknowledgments

This work was supported in part by NSF grant numbers 2239020, 1565570, and 2402873, in part by ACE, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA, in part by the Office of Naval Research contract number N000142412612, and in part by the Center for Intelligent Information Retrieval. Any opinions, findings, conclusions, and recommendations expressed in this material are those of the authors and do not necessarily reflect those of the sponsors. We thank Jae-sun Seo and Yuan Liao from Cornell University for their help in synthesizing the near-memory accelerators on 16nm TSMC technology.

A Artifact Appendix

A.1 Abstract

This appendix describes two artifacts: 1–The cycle-approximate simulator for IKS, which models IKS using timing data gathered from RTL synthesis. 2–FAISS modified for fast ENNS on Intel CPUs. All artifacts are available via Github.

A.2 Artifact check-list

- **Simulator:** https://github.com/architecture-research-group/iks_simulator
- **Optimized Faiss:** https://github.com/architecture-research-group/ae-asplo25-iks-faiss/tree/main
- **Compilation:** Please refer to each program's repository.
- **OS requirement:** Modern Linux kernel

- **Hardware requirement:** Intel® 4th Gen Xeon Scalable Processors or newer, with AMX equipped and enabled.
- **Software requirement:** Intel® MKL Installed
- **Publicly available?:** Yes.

References

[1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *Microsoft Research Blog* (September 2023). https://doi.org/10.48550/arXiv.2308.16369

[2] Yeonchan Ahn, Sang-Goo Lee, Junho Shim, and Jaehui Park. 2022. Retrieval-Augmented Response Generation for Knowledge-Grounded Conversation in the Wild. *IEEE Access* 10 (2022), 131374–131385. https://doi.org/10.1109/ACCESS.2022.3228964

[3] Meta AI. 2024. Llama 3. Online; accessed 2024-12-13. https://llama.meta.com/llama3/

[4] Mohammad Alian, Seung Won Min, Hadi Asgharimoghaddam, Ashutosh Dhar, Dong Kai Wang, Thomas Roewer, Adam McPadden, Oliver O'Halloran, Deming Chen, Jinjun Xiong, et al. 2018. Application-Transparent Near-Memory Processing Architecture with Memory Channel Network. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 802–814. https://doi.org/10.1109/MICRO.2018.00070

[5] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2023. Llm in a flash: Efficient large language model inference with limited memory. *arXiv preprint arXiv:2312.11514* (2023). https://doi.org/10.48550/arXiv.2312.11514

[6] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374. https://doi.org/10.48550/arXiv.1807.05614

[7] Artem Babenko and Victor Lempitsky. 2012. The inverted multi-index. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. 3069–3076. https://doi.org/10.1109/CVPR.2012.6248038

[8] Giovanni Bonetta, Rossella Cancelliere, Ding Liu, and Paul Vozila. 2021. Retrieval-Augmented Transformer-XL for Close-Domain Dialog Generation. *The International FLAIRS Conference Proceedings* 34, 1 (April 2021). https://doi.org/10.32473/flairs.v34i1.128369

[9] Francesco Busolin, Claudio Lucchese, Franco Maria Nardini, Salvatore Orlando, Raffaele Perego, and Salvatore Trani. 2024. Early Exit Strategies for Approximate k-NN Search in Dense Retrieval. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management* (Boise, ID, USA) *(CIKM '24)*. Association for Computing Machinery, New York, NY, USA, 3647–3652. https://doi.org/10.1145/3627673.3679903

[10] Deng Cai, Yan Wang, Wei Bi, Zhaopeng Tu, Xiaojiang Liu, Wai Lam, and Shuming Shi. 2019. Skeleton-to-Response: Dialogue Generation Guided by Retrieval Memory. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 1219–1228. https://doi.org/10.18653/v1/N19-1124

[11] Wenhu Chen, Hexiang Hu, Xi Chen, Pat Verga, and William Cohen. 2022. MuRAG: Multimodal Retrieval-Augmented Generator for Open Question Answering over Images and Text. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 5558–5570. https://doi.org/10.18653/v1/2022.emnlp-main.375

[12] Wenhu Chen, Hexiang Hu, Chitwan Saharia, and William W. Cohen. 2022. Re-Imagen: Retrieval-Augmented Text-to-Image Generator. *ArXiv* abs/2209.14491 (2022). https://api.semanticscholar.org/CorpusID:252596087

[13] William J. Dally, Yatish Turakhia, and Song Han. 2020. Domain-specific hardware accelerators. *Commun. ACM* 63, 7 (2020), 48–57. https://doi.org/10.1145/3361682

[14] Feyza Duman Keles, Pruthuvi Mahesakya Wijewardena, and Chinmay Hegde. 2023. On The Computational Complexity of Self-Attention. In *Proceedings of The 34th International Conference on Algorithmic Learning Theory (Proceedings of Machine Learning Research, Vol. 201)*, Shipra Agrawal and Francesco Orabona (Eds.). PMLR, 597–619. https://proceedings.mlr.press/v201/duman-keles23a.html

[15] Zhengcong Fei. 2021. Memory-Augmented Image Captioning. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 2 (May 2021), 1317–1324. https://doi.org/10.1609/aaai.v35i2.16220

[16] Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, Adam Hutchin, Utku Diril, Krishnakumar Nair, Ehsan K. Aredestani, Martin Schatz, Yuchen Hao, Rakesh Komuravelli, Kunming Ho, Sameer Abu Asal, Joe Shajrawi, Kevin Quinn, Nagesh Sreedhara, Pankaj Kansal, Willie Wei, Dheepak Jayaraman, Linda Cheng, Pritam Chopda, Eric Wang, Ajay Bikumandla, Arun Karthik Sengottuvel, Krishna Thottempudi, Ashwin Narasimha, Brian Dodds, Cao Gao, Jiyuan Zhang, Mohammed Al-Sanabani, Ana Zehtabioskuie, Jordan Fix, Hangchen Yu, Richard Li, Kaustubh Gondkar, Jack Montgomery, Mike Tsai, Saritha Dwarakapuram, Sanjay Desai, Nili Avidan, Poorvaja Ramani, Karthik Narayanan, Ajit Mathews, Sethu Gopal, Maxim Naumov, Vijay Rao, Krishna Noru, Harikrishna Reddy, Prahlad Venkatapuram, and Alexis Bjorlin. 2023. MTIA: First Generation Silicon Targeting Meta's Recommendation Systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 80, 13 pages. https://doi.org/10.1145/3579371.3589348

[17] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2023. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934* (2023). https://doi.org/10.48550/arXiv.2311.04934

[18] Jiatao Gu, Yong Wang, Kyunghyun Cho, and Victor OK Li. 2018. Search engine guided neural machine translation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32. https://doi.org/10.1609/aaai.v32i1.12013

[19] Liangke Gui, Borui Wang, Qiuyuan Huang, Alexander Hauptmann, Yonatan Bisk, and Jianfeng Gao. 2022. KAT: A Knowledge Augmented Transformer for Vision-and-Language. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, Seattle, United States, 956–968. https://doi.org/10.18653/v1/2022.naacl-main.70

[20] Tatsunori B. Hashimoto, Kelvin Guu, Yonatan Oren, and Percy Liang. 2018. A Retrieve-and-Edit Framework for Predicting Structured Outputs. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 10073–10083. https://dl.acm.org/doi/10.5555/3327546.3327670

[21] Qiuxiang He, Guoping Huang, Qu Cui, Li Li, and Lemao Liu. 2021. Fast and Accurate Neural Machine Translation with Translation Memory. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 3170–3180.

https://doi.org/10.18653/v1/2021.acl-long.246

[22] Sebastian Hofstätter, Jiecao Chen, Karthik Raman, and Hamed Zamani. 2023. FiD-Light: Efficient and Effective Retrieval-Augmented Text Generation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Taipei, Taiwan) *(SIGIR '23)*. Association for Computing Machinery, New York, NY, USA, 1437–1447. https://doi.org/10.1145/3539618.3591687

[23] Mark Horowitz. 2014. 1.1 Computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. 10–14. https://doi.org/10.1109/ISSCC.2014.6757323

[24] Mohamed Assem Ibrahim, Onur Kayiran, Yasuko Eckert, Gabriel H Loh, and Adwait Jog. 2021. Analyzing and leveraging decoupled L1 caches in GPUs. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 467–478. https://doi.org/10.1109/HPCA51647.2021.00047

[25] Gautier Izacard and Edouard Grave. 2021. Distilling Knowledge from Reader to Retriever for Question Answering. In *International Conference on Learning Representations*. https://openreview.net/forum?id=NTEz-6wysdb

[26] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty (Eds.). Association for Computational Linguistics, Online, 874–880. https://doi.org/10.18653/v1/2021.eacl-main.74

[27] Hervé Jégou, Matthijs Douze, and Jeff Johnson. 2017. Faiss: A Library for Efficient Similarity Search. Engineering at Meta. https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/ Accessed: 2024-12-13.

[28] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[29] Nikhil Kandpal, Eric Wallace, and Colin Raffel. 2022. Deduplicating training data mitigates privacy risks in language models. In *International Conference on Machine Learning*. PMLR, 10697–10707. https://proceedings.mlr.press/v162/kandpal22a.html

[30] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6769–6781. https://doi.org/10.18653/v1/2020.emnlp-main.550

[31] Jungo Kasai, Keisuke Sakaguchi, Yoichi Takahashi, Ronan Le Bras, Akari Asai, Xinyan Velocity Yu, Dragomir Radev, Noah A. Smith, Yejin Choi, and Kentaro Inui. 2024. REALTIME QA: what's the answer right now?. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2130, 19 pages. https://dl.acm.org/doi/10.5555/3666122.3668252

[32] Amirhossein Kazemnejad, Mohammadreza Salehi, and Mahdieh Soleymani Baghshah. 2020. Paraphrase Generation by Learning How to Edit from Samples. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 6010–6021. https://doi.org/10.18653/v1/2020.acl-main.535

[33] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. 2022. Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM. *IEEE Micro* 42, 1 (2022), 116–127. https://doi.org/10.1109/MM.2021.3097700

[34] Ben Keller, Rangharajan Venkatesan, Steve Dai, Stephen G Tell, Brian Zimmer, Charbel Sakr, William J Dally, C Thomas Gray, and Brucek Khailany. 2023. A 95.6-TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization in 5 nm. *IEEE Journal of Solid-State Circuits* 58, 4 (2023), 1129–1141. https://doi.org/https://doi.org/10.1109/VLSITechnologyandCir46769.2022.9830277

[35] Jin Hyun Kim, Shin-Haeng Kang, Sukhan Lee, Hyeonsu Kim, Yuhwan Ro, Seungwon Lee, David Wang, Jihyun Choi, Jinin So, YeonGon Cho, JoonHo Song, Jeonghyeon Cho, Kyomin Sohn, and Nam Sung Kim. 2022. Aquabolt-XL HBM2-PIM, LPDDR5-PIM With In-Memory Processing, and AXDIMM With Acceleration Buffer. *IEEE Micro* 42, 3 (2022), 20–30. https://doi.org/10.1109/MM.2022.3164651

[36] To Eun Kim, Alireza Salemi, Andrew Drozdov, Fernando Diaz, and Hamed Zamani. 2024. Retrieval-Enhanced Machine Learning: Synthesis and Opportunities. *arXiv* (2024). https://doi.org/10.48550/arXiv.2407.12982 arXiv:2407.12982.

[37] Ishita Kumar, Snigdha Viswanathan, Sushrita Yerra, Alireza Salemi, Ryan A. Rossi, Franck Dernoncourt, Hanieh Deilamsalehy, Xiang Chen, Ruiyi Zhang, Shubham Agarwal, Nedim Lipka, Chien Van Nguyen, Thien Huu Nguyen, and Hamed Zamani. 2024. LongLaMP: A Benchmark for Personalized Long-form Text Generation. *arXiv* (2024). https://doi.org/10.48550/arXiv.2407.11016 arXiv:2407.11016.

[38] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: a Benchmark for Question Answering Research. *Transactions of the Association of Computational Linguistics* (2019). https://doi.org/10.1162/tacl_a_00276

[39] Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent retrieval for weakly supervised open domain question answering. *arXiv preprint arXiv:1906.00300* (2019). https://doi.org/10.48550/arXiv.1906.00300

[40] Yejin Lee, Hyunji Choi, Sunhong Min, Hyunseung Lee, Sangwon Beak, Dawoon Jeong, Jae W. Lee, and Tae Jun Ham. 2022. ANNA: Specialized Architecture for Approximate Nearest Neighbor Search. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 169–183. https://doi.org/10.1109/HPCA53966.2022.00021

[41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS'20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages. https://doi.org/10.5555/3495724.3496517

[42] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 2539–2554. https://doi.org/10.1145/3318464.3380600

[43] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 574–587. https://doi.org/10.1145/3575693.3578835

[44] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. 2022. A Survey on Retrieval-Augmented Text Generation. *ArXiv* abs/2202.01110 (2022). https://api.semanticscholar.org/CorpusID:246472929

[45] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2020), 1475–1488. https://doi.org/10.1109/TKDE.2019.2909204

[46] Pierre Lienhart. 2024. LLM Inference Series: 4. KV caching, a deeper look. *Pierre Leinhart (Medium)* (Jan 2024). https://medium.com/@plienhar/llm-inference-series-4-kv-caching-a-deeper-look-4ba9a77746c8

[47] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. https://aclanthology.org/W04-1013

[48] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978* (2023). https://doi.org/10.48550/arXiv.2306.00978

[49] Shangqing Liu, Yu Chen, Xiaofei Xie, Jing Kai Siow, and Yang Liu. 2021. Retrieval-Augmented Generation for Code Summarization via Hybrid GNN. In *International Conference on Learning Representations*. https://openreview.net/forum?id=zv-typ1gPxA

[50] Locuza. 2022. Die Analysis: Samsung Exynos 2200 with RDNA2 Graphics. Online; accessed 2024-12-13. https://locuza.substack.com/p/die-analysis-samsung-exynos-2200

[51] Gabriel H. Loh, Natalie Enright Jerger, Ajaykumar Kannan, and Yasuko Eckert. 2015. Interconnect-Memory Challenges for Multi-chip, Silicon Interposer Systems. In *Proceedings of the 2015 International Symposium on Memory Systems* (Washington DC, DC, USA) *(MEMSYS '15)*. Association for Computing Machinery, New York, NY, USA, 3–10. https://doi.org/10.1145/2818950.2818951

[52] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (apr 2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[53] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 932–949. https://doi.org/10.1145/3620666.3651335

[54] Timothy Prickett Morgan. 2024. He Who Can Pay Top Dollar For HBM Memory Controls AI Training. *The Next Platform* (2024). https://www.nextplatform.com/2024/02/27/he-who-can-pay-top-dollar-for-hbm-memory-controls-ai-training/ Accessed: 2024-06-23.

[55] Samuel Naffziger, Kevin Lepak, Milam Paraschou, and Mahesh Subramony. 2020. 2.2 AMD Chiplet Architecture for High-Performance Server and Desktop Products. In *2020 IEEE International Solid- State Circuits Conference - (ISSCC)*. 44–45. https://doi.org/10.1109/ISSCC19947.2020.9063103

[56] OpenAI. 2023. ChatGPT plugins. *OpenAI Blog* (2023). https://openai.com/blog/chatgpt-plugins

[57] Sang-Soo Park, KyungSoo Kim, Jinin So, Jin Jung, Jonggeon Lee, Kyoungwan Woo, Nayeon Kim, Younghyun Lee, Hyungyo Kim, Yongsuk Kwon, Jinhyun Kim, Jieun Lee, YeonGon Cho, Yongmin Tai, Jeonghyeon Cho, Hoyoung Song, Jung Ho Ahn, and Nam Sung Kim. 2024. An LPDDR-based CXL-PNM Platform for TCO-efficient Inference of Transformer-based Large Language Models. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 970–982. https://doi.org/10.1109/HPCA57654.2024.00078

[58] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval Augmented Code Generation and Summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Punta Cana, Dominican Republic, 2719–2734. https://doi.org/10.18653/v1/2021.findings-emnlp.232

[59] Dylan Patel. 2022. Apple M2 Die Shot and Architecture Analysis – Big Cost Increase And A15 Based IP. *SemiAnalysis* (June 2022). https://www.semianalysis.com/p/apple-m2-die-shot-and-architecture

[60] Dylan Patel and Jeremie Eliahou Ontiveros. 2024. CXL Is Dead In The AI Era. Online; accessed 2024-12-13. https://www.semianalysis.com/p/cxl-is-dead-in-the-ai-era

[61] N. Patel, A. Mamandipoor, M. Nouri, and M. Alian. 2024. SmartDIMM: In-Memory Acceleration of Upper Layer Protocols. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 312–329. https://doi.org/10.1109/HPCA57654.2024.00032

[62] Neel Patel, Amin Mamandipoor, Derrick Quinn, and Mohammad Alian. 2023. XFM: Accelerated Software-Defined Far Memory. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 769–783. https://doi.org/10.1145/3613424.3623776

[63] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting . In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 118–132. https://doi.org/10.1109/ISCA59077.2024.00019

[64] Hao Peng, Ankur Parikh, Manaal Faruqui, Bhuwan Dhingra, and Dipanjan Das. 2019. Text Generation with Exemplar-based Adaptive Decoding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 2555–2565. https://doi.org/10.18653/v1/N19-1263

[65] Fabio Petroni, Aleksandra Piktus, and Angela Fan. 2020. Introducing KILT, a new unified benchmark for knowledge-intensive NLP tasks. Online; accessed 2024-11-22. https://ai.meta.com/blog/introducing-kilt-a-new-unified-benchmark-for-knowledge-intensive-nlp-tasks/

[66] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. KILT: a Benchmark for Knowledge Intensive Language Tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 2523–2544. https://doi.org/10.18653/v1/2021.naacl-main.200

[67] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur,

Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, Online, 5835–5847. https://doi.org/10.18653/v1/2021.naacl-main.466

[68] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *The Journal of Machine Learning Research* 21, 1, Article 140 (jan 2020), 67 pages. https://doi.org/10.5555/3455716.3455856

[69] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, Jian Su, Kevin Duh, and Xavier Carreras (Eds.). Association for Computational Linguistics, Austin, Texas, 2383–2392. https://doi.org/10.18653/v1/D16-1264

[70] Rita Ramos, Desmond Elliott, and Bruno Martins. 2023. Retrieval-augmented Image Captioning. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, Andreas Vlachos and Isabelle Augenstein (Eds.). Association for Computational Linguistics, Dubrovnik, Croatia, 3666–3681. https://doi.org/10.18653/v1/2023.eacl-main.266

[71] Alireza Salemi, Juan Altmayer Pizzorno, and Hamed Zamani. 2023. A Symmetric Dual Encoding Dense Retrieval Framework for Knowledge-Intensive Visual Question Answering. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Taipei, Taiwan) *(SIGIR '23)*. Association for Computing Machinery, New York, NY, USA, 110–120. https://doi.org/10.1145/3539618.3591629

[72] Alireza Salemi, Surya Kallumadi, and Hamed Zamani. 2024. Optimization Methods for Personalizing Large Language Models through Retrieval Augmentation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) *(SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 752–762. https://doi.org/10.1145/3626772.3657783

[73] Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2024. LaMP: When Large Language Models Meet Personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 7370–7392. https://doi.org/10.18653/v1/2024.acl-long.399

[74] Alireza Salemi, Mahta Rafiee, and Hamed Zamani. 2023. Pre-Training Multi-Modal Dense Retrievers for Outside-Knowledge Visual Question Answering. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval* (Taipei, Taiwan) *(ICTIR '23)*. Association for Computing Machinery, New York, NY, USA, 169–176. https://doi.org/10.1145/3578337.3605137

[75] Alireza Salemi and Hamed Zamani. 2024. Comparing Retrieval-Augmentation and Parameter-Efficient Fine-Tuning for Privacy-Preserving Personalization of Large Language Models. arXiv:2409.09510 [cs.CL] https://arxiv.org/abs/2409.09510

[76] Alireza Salemi and Hamed Zamani. 2024. Evaluating Retrieval Quality in Retrieval-Augmented Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) *(SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 2395–2400. https://doi.org/10.1145/3626772.3657957

[77] Alireza Salemi and Hamed Zamani. 2024. Learning to Rank for Multiple Retrieval-Augmented Models through Iterative Utility Maximization. arXiv:2410.09942 [cs.CL] https://arxiv.org/abs/2410.09942

[78] Alireza Salemi and Hamed Zamani. 2024. Towards a Search Engine for Machines: Unified Ranking for Multiple Retrieval-Augmented Large Language Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) *(SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 741–751. https://doi.org/10.1145/3626772.3657733

[79] Satyabrata Sarangi and Bevan Baas. 2021. DeepScaleTool: A Tool for the Accurate Estimation of Technology Scaling in the Deep-Submicron Era. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. https://doi.org/10.1109/ISCAS51556.2021.9401196

[80] Sara Sarto, Marcella Cornia, Lorenzo Baraldi, and Rita Cucchiara. 2022. Retrieval-Augmented Transformer for Image Captioning. In *Proceedings of the 19th International Conference on Content-Based Multimedia Indexing* (Graz, Austria) *(CBMI '22)*. Association for Computing Machinery, New York, NY, USA, 1–7. https://doi.org/10.1145/3549555.3549585

[81] Henry N. Schuh, Arvind Krishnamurthy, David Culler, Henry M. Levy, Luigi Rizzo, Samira Khan, and Brent E. Stephens. 2024. CC-NIC: a Cache-Coherent Interface to the NIC. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 52–68. https://doi.org/10.1145/3617232.3624868

[82] Haihao Shen, Hanwen Chang, Bo Dong, Yu Luo, and Hengyu Meng. 2023. Efficient llm inference on cpus. *arXiv preprint* (2023). https://doi.org/10.48550/arXiv.2311.00502

[83] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval Augmentation Reduces Hallucination in Conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Punta Cana, Dominican Republic, 3784–3803. https://doi.org/10.18653/v1/2021.findings-emnlp.320

[84] Joonseop Sim, Soohong Ahn, Taeyoung Ahn, Seungyong Lee, Myunghyun Rhee, Jooyoung Kim, Kwangsik Shin, Donguk Moon, Euiseok Kim, and Kyoung Park. 2023. Computational CXL-Memory Solution for Accelerating Memory-Intensive Applications. *IEEE Computer Architecture Letters* 22, 1 (2023), 5–8. https://doi.org/10.1109/LCA.2022.3226482

[85] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kaluarachchi, Rajib Rana, and Suranga Nanayakkara. 2023. Improving the Domain Adaptation of Retrieval Augmented Generation (RAG) Models for Open Domain Question Answering. *Transactions of the Association for Computational Linguistics* 11 (01 2023), 1–17. https://doi.org/10.1162/tacl_a_00530 arXiv:https://direct.mit.edu/tacl/article-pdf/doi/10.1162/tacl_a_00530/2067834/tacl_a_00530.pdf

[86] Heidi Steen and Dan Wahlin. 2023. Retrieval Augumented Generation Overview. *Microsoft Learn* (2023). https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview

[87] Jovan Stojkovic, Esha Choukse, Chaojie Zhang, Inigo Goiri, and Josep Torrellas. 2024. Towards Greener LLMs: Bringing Energy-Efficiency to the Forefront of LLM Inference. *arXiv preprint arXiv:2403.20306* (2024). arXiv:2403.20306 [cs.AI] https://doi.org/10.48550/arXiv.2403.20306

[88] Lisa T. Su, Samuel Naffziger, and Mark Papermaster. 2017. Multi-chip technologies to unleash computing performance gains over the next decade. In *2017 IEEE International Electron Devices Meeting (IEDM)*. 1.1.1–1.1.8. https://doi.org/10.1109/IEDM.2017.8268306

[89] Yixuan Su, David Vandyke, Simon Baker, Yan Wang, and Nigel Collier. 2021. Keep the Primary, Rewrite the Secondary: A Two-Stage Approach for Paraphrase Generation. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 560–569. https://doi.org/10.18653/v1/2021.findings-acl.50

[90] Gemini Team. 2024. Gemini: A Family of Highly Capable Multimodal Models. *arXiv* (2024). arXiv:2312.11805 [cs.CL] https://doi.org/10.48550/arXiv.2312.11805

[91] David Thulke, Nico Daheim, Christian Dugast, and Hermann Ney. 2021. Efficient retrieval augmented generation from unstructured knowledge for task-oriented dialog. *arXiv preprint arXiv:2102.04643* (2021). https://doi.org/10.48550/arXiv.2102.04643

[92] Zhiliang Tian, Wei Bi, Xiaopeng Li, and Nevin L. Zhang. 2019. Learning to Abstract for Memory-augmented Conversational Response Generation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Anna Korhonen, David Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, Florence, Italy, 3816–3825. https://doi.org/10.18653/v1/P19-1371

[93] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[94] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 14, 11 (jul 2021), 1964–1978. https://doi.org/10.14778/3476249.3476255

[95] Yitu Wang, Shiyu Li, Qilin Zheng, Linghao Song, Zongwang Li, Andrew Chang, Hai "Helen" Li, and Yiran Chen. 2024. NDSEARCH: Accelerating Graph-Traversal-Based Approximate Nearest Neighbor Search through Near Data Processing. In *Proceedings of the 39th Annual International Symposium on Computer Architecture*. arXiv:2312.03141 https://doi.org/10.48550/arXiv.2312.03141

[96] Zelin Wang, Ping Gong, Yibo Zhang, Jihao Gu, and Xuanyuan Yang. 2023. Retrieval-Augmented Knowledge-Intensive Dialogue. In *Natural Language Processing and Chinese Computing*, Fei Liu, Nan Duan, Qingting Xu, and Yu Hong (Eds.). Springer Nature Switzerland, Cham, 16–28. https://doi.org/10.48550/arXiv.2005.11401

[97] Jason Weston, Emily Dinan, and Alexander Miller. 2018. Retrieve and Refine: Improved Sequence Generation Models For Dialogue. In *Proceedings of the 2018 EMNLP Workshop SCAI: The 2nd International Workshop on Search-Oriented Conversational AI*, Aleksandr Chuklin, Jeff Dalton, Julia Kiseleva, Alexey Borisov, and Mikhail Burtsev (Eds.). Association for Computational Linguistics, Brussels, Belgium, 87–92. https://doi.org/10.18653/v1/W18-5713

[98] WikiChip. 2024. Mask / Reticle. Online; accessed 2024-12-13. https://en.wikichip.org/wiki/mask

[99] Yu Wu, Furu Wei, Shaohan Huang, Yunli Wang, Zhoujun Li, and Ming Zhou. 2019. Response Generation by Context-Aware Prototype Editing. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 01 (Jul. 2019), 7281–7288. https://doi.org/10.1609/aaai.v33i01.33017281

[100] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099. https://doi.org/10.5555/3618408.3619993

[101] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=NG7sS51zVF

[102] Jitao Xu, Josep Crego, and Jean Senellart. 2020. Boosting Neural Machine Translation with Similar Translations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 1580–1590. https://doi.org/10.18653/v1/2020.acl-main.144

[103] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. *arXiv* (2024). https://doi.org/10.48550/arXiv.2401.15884 arXiv:2401.15884 [cs.CL]

[104] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487* (2023). https://doi.org/10.48550/arXiv.2304.04487

[105] Yifan Yuan, Jinghan Huang, Yan Sun, Tianchen Wang, Jacob Nelson, Dan R. K. Ports, Yipeng Wang, Ren Wang, Charlie Tai, and Nam Sung Kim. 2023. Rambda: RDMA-driven Acceleration Framework for Memory-intensive µs-scale Datacenter Applications. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 499–515. https://doi.org/10.1109/HPCA56546.2023.10071127

[106] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. 2024. The Shift from Models to Compound AI Systems. Online; accessed 2024-12-13. https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/

[107] Hamed Zamani and Michael Bendersky. 2024. Stochastic RAG: End-to-End Retrieval-Augmented Generation through Expected Utility Maximization. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) *(SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 2641–2646. https://doi.org/10.1145/3626772.3657923

[108] Hamed Zamani, Fernando Diaz, Mostafa Dehghani, Donald Metzler, and Michael Bendersky. 2022. Retrieval-Enhanced Machine Learning. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Madrid, Spain) *(SIGIR '22)*. Association for Computing Machinery, New York, NY, USA, 2875–2886. https://doi.org/10.1145/3477495.3531722

[109] Jingyi Zhang, Masao Utiyama, Eiichro Sumita, Graham Neubig, and Satoshi Nakamura. 2018. Guiding Neural Machine Translation with Retrieved Translation Pieces. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, Marilyn Walker, Heng Ji, and Amanda Stent (Eds.). Association for Computational Linguistics, New Orleans, Louisiana, 1325–1335. https://doi.org/10.18653/v1/N18-1120

[110] Yunan Zhang, Shige Liu, and Jianguo Wang. [n. d.]. Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL. *Preprint* ([n. d.]). https://www.cs.purdue.edu/homes/csjgwang/pubs/ICDE24_VecDB.pdf Accepted for publication in Proceedings of the International Conference on Data Engineering (ICDE).

[111] Zhe Zhou, Cong Li, Fan Yang, and Guangyu Sun. 2023. DIMM-Link: Enabling Efficient Inter-DIMM Communication for Near-Memory Processing. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 302–316. https://doi.org/10.1109/HPCA56546.2023.10071005 ISSN: 2378-203X.

[112] Yun Zhu, Jia-Chen Gu, Caitlin Sikora, Ho Ko, Yinxiao Liu, Chu-Cheng Lin, Lei Shu, Liangchen Luo, Lei Meng, Bang Liu, and Jindong Chen. 2024. Accelerating Inference of Retrieval-Augmented Generation via Sparse Context Selection. *arXiv* (2024). https://doi.org/10.48550/arXiv.2405.16178